

1. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.
To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

PART 1 — Tokenizer

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 100
#define MAX_TOKEN_LENGTH 100

/* Token types */
typedef enum
{
    TOKEN_WORD,
    TOKEN_PIPE,
    TOKEN_INPUT,
    TOKEN_OUTPUT,
    TOKEN_APPEND,
    TOKEN_BACKGROUND,
    TOKEN_END
} TokenType;

/* Token structure */
typedef struct
{
    TokenType type;
    char value[MAX_TOKEN_LENGTH];
} Token;

/* Convert token type to string */
const char *token_type_name(TokenType type)
{
    switch (type)
    {
        case TOKEN_WORD:
            return "WORD";
        case TOKEN_PIPE:
            return "PIPE";
        case TOKEN_INPUT:
            return "INPUT";
        case TOKEN_OUTPUT:
            return "OUTPUT";
        case TOKEN_APPEND:
            return "APPEND";
        case TOKEN_BACKGROUND:
            return "BACKGROUND";
        case TOKEN_END:
            return "END";
        default:
            return "UNKNOWN";
    }
}

```

```

/* Add a token */
void add_token(Token tokens[], int *count, TokenType type, const char *value)
{
    if (*count >= MAX_TOKENS - 1)
    {
        printf("Error: Too many tokens.\n");
        return;
    }

    tokens[*count].type = type;

    strncpy(tokens[*count].value, value, MAX_TOKEN_LENGTH - 1);
    tokens[*count].value[MAX_TOKEN_LENGTH - 1] = '\0';

    (*count)++;
}

/* Tokenize input */
int tokenize(char *input, Token tokens[])
{
    int i = 0;
    int count = 0;

    while (input[i] != '\0')
    {
        /* Ignore whitespace */
        if (isspace((unsigned char)input[i]))
        {
            i++;
            continue;
        }

        /* Pipe */
        if (input[i] == '|')
        {
            add_token(tokens, &count, TOKEN_PIPE, "|");
            i++;
        }

        /* Input redirection */
        else if (input[i] == '<')
        {
            add_token(tokens, &count, TOKEN_INPUT, "<");
            i++;
        }

        /* Output redirection */
        else if (input[i] == '>')
        {
            if (input[i + 1] == '>')
            {
                add_token(tokens, &count, TOKEN_APPEND, ">>");
                i += 2;
            }
        }
    }
}

```

```

        else
        {
            add_token(tokens, &count, TOKEN_OUTPUT, ">");
            i++;
        }
    }

    /* Background */
    else if (input[i] == '&')
    {
        add_token(tokens, &count, TOKEN_BACKGROUND, "&");
        i++;
    }

    /* Quoted string */
    else if (input[i] == '"' || input[i] == '\')
    {
        char quote = input[i];
        char word[MAX_TOKEN_LENGTH];
        int j = 0;

        i++;

        while (input[i] != '\0' && input[i] != quote)
        {
            if (j < MAX_TOKEN_LENGTH - 1)
            {
                word[j++] = input[i];
            }

            i++;
        }

        word[j] = '\0';

        if (input[i] == quote)
        {
            i++;
        }
        else
        {
            printf("Error: Unclosed quote.\n");
            return -1;
        }

        add_token(tokens, &count, TOKEN_WORD, word);
    }

    /* Normal word */
    else
    {
        char word[MAX_TOKEN_LENGTH];
        int j = 0;

```

```

        while (input[i] != '\0' &&
               !isspace((unsigned char)input[i]) &&
               input[i] != '|' &&
               input[i] != '<' &&
               input[i] != '>' &&
               input[i] != '&')
        {
            if (j < MAX_TOKEN_LENGTH - 1)
            {
                word[j++] = input[i];
            }

            i++;
        }

        word[j] = '\0';

        if (j > 0)
        {
            add_token(tokens, &count, TOKEN_WORD, word);
        }
    }

    add_token(tokens, &count, TOKEN_END, "");

    return count;
}

int main()
{
    char input[500];
    Token tokens[MAX_TOKENS];

    printf("=== Shell Tokenizer ===\n");
    printf("Enter a command:\n\n");

    printf("shell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Handle empty command */
    if (strlen(input) == 0)
    {
        printf("Empty command.\n");
        return 0;
    }
}

```

```

}

int count = tokenize(input, tokens);

if (count < 0)
{
    return 1;
}

printf("\nTokens generated:\n");
printf("-----\n");

for (int i = 0; i < count; i++)
{
    printf("%-12s : %s\n",
           token_type_name(tokens[i].type),
           tokens[i].value);

    if (tokens[i].type == TOKEN_END)
    {
        break;
    }
}

return 0;

```

```
adithya@LAPTOP-07A78KDI:~/OS$ gcc tokenizer.c -o tokenizer
adithya@LAPTOP-07A78KDI:~/OS$ ./tokenizer
=== Shell Tokenizer ===
Enter a command:

shell> ls -l | grep ".c"

Tokens generated:
-----
WORD      : ls
WORD      : -l
PIPE       : |
WORD      : grep
WORD      : .c
END        :
adithya@LAPTOP-07A78KDI:~/OS$ |
```

```
adithya@LAPTOP-07A78KDI:~/OS$ ./tokenizer
=== Shell Tokenizer ===
Enter a command:

shell> ls -l | grep ".c"

Tokens generated:
-----
WORD      : ls
WORD      : -l
PIPE       : |
WORD      : grep
WORD      : .c
END        :
adithya@LAPTOP-07A78KDI:~/OS$
```

```
adithya@LAPTOP-07A78KDI:~/OS$ ./tokenizer
=== Shell Tokenizer ===
Enter a command:

shell> cat < input.txt

Tokens generated:
-----
WORD      : cat
INPUT      : <
WORD      : input.txt
END        :
adithya@LAPTOP-07A78KDI:~/OS$ |
```

```
adithya@LAPTOP-07A78KDI:~/OS$ ./tokenizer
=== Shell Tokenizer ===
Enter a command:

shell> ls > output.txt

Tokens generated:
-----
WORD      : ls
OUTPUT     : >
WORD      : output.txt
END        :
adithya@LAPTOP-07A78KDI:~/OS$ |
```

```
adithya@LAPTOP-07A78KDI:~/OS$ sleep 10 &
[1] 1960
```

```
adithya@LAPTOP-07A78KDI:~/OS$ ./tokenizer
=== Shell Tokenizer ===
Enter a command:

shell> echo "hello
Error: Unclosed quote.
[1]+  Done                  sleep 10
adithya@LAPTOP-07A78KDI:~/OS$ |
```

PART 2 — Parser and Parse Tree

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 100
#define MAX_LENGTH 100

typedef enum
{
    WORD,
    PIPE,
    INPUT,
    OUTPUT,
    APPEND,
    BACKGROUND,
    END
} TokenType;

typedef struct
{
    TokenType type;
    char value[MAX_LENGTH];
} Token;

/* Execution command structure */
typedef struct Command
{
    char *arguments[MAX_TOKENS];
    int argument_count;

    char *input_file;
    char *output_file;

    int append;
    int background;

    struct Command *next;
} Command;

/* Token type name */
const char *token_name(TokenType type)
{
    switch (type)
    {
        case WORD:
            return "WORD";

        case PIPE:
            return "PIPE";

        case INPUT:
            return "INPUT";

        case OUTPUT:
            return "OUTPUT";

        case APPEND:
            return "APPEND";

        case BACKGROUND:
            return "BACKGROUND";

        case END:
            return "END";

        default:
            return "UNKNOWN";
    }
}

/* Simple tokenizer */
int tokenize(char *input, Token tokens[])
{
    int i = 0;
    int count = 0;

    while (input[i] != '\0')
```

```

{
    if (isspace((unsigned char)input[i]))
    {
        i++;
        continue;
    }

    if (input[i] == '|')
    {
        tokens[count].type = PIPE;
        strcpy(tokens[count].value, "|");
        count++;
        i++;
    }
    else if (input[i] == '<')
    {
        tokens[count].type = INPUT;
        strcpy(tokens[count].value, "<");
        count++;
        i++;
    }
    else if (input[i] == '>')
    {
        if (input[i + 1] == '>')
        {
            tokens[count].type = APPEND;
            strcpy(tokens[count].value, ">>");
            count++;
            i += 2;
        }
        else
        {
            tokens[count].type = OUTPUT;
            strcpy(tokens[count].value, ">");
            count++;
            i++;
        }
    }
    else if (input[i] == '&')
    {
        tokens[count].type = BACKGROUND;
        strcpy(tokens[count].value, "&");
        count++;
        i++;
    }
    else
    {
        char word[MAX_LENGTH];
        int j = 0;

        while (input[i] != '\0' &&
            !isspace((unsigned char)input[i]) &&
            input[i] != '|' &&
            input[i] != '<' &&
            input[i] != '>' &&
            input[i] != '&')
        {
            if (j < MAX_LENGTH - 1)
            {
                word[j++] = input[i];
            }

            i++;
        }

        word[j] = '\0';

        tokens[count].type = WORD;
        strcpy(tokens[count].value, word);
        count++;
    }
}

tokens[count].type = END;
tokens[count].value[0] = '\0';

return count;
}

/* Create new command */
Command *create_command()

```

```

{
    Command *cmd = malloc(sizeof(Command));

    if (cmd == NULL)
    {
        perror("malloc");
        exit(1);
    }

    cmd->argument_count = 0;
    cmd->input_file = NULL;
    cmd->output_file = NULL;
    cmd->append = 0;
    cmd->background = 0;
    cmd->next = NULL;

    return cmd;
}

/* Free command list */
void free_commands(Command *head)
{
    Command *temp;

    while (head != NULL)
    {
        temp = head;
        head = head->next;

        for (int i = 0; i < temp->argument_count; i++)
        {
            free(temp->arguments[i]);
        }

        free(temp->input_file);
        free(temp->output_file);
        free(temp);
    }
}

/* Parse tokens */
Command *parse(Token tokens[], int count)
{
    Command *head = NULL;
    Command *current = NULL;

    int i = 0;

    if (count == 0 || tokens[0].type == END)
    {
        printf("Error: Empty command.\n");
        return NULL;
    }

    head = create_command();
    current = head;

    while (i < count && tokens[i].type != END)
    {
        /* WORD */
        if (tokens[i].type == WORD)
        {
            current->arguments[current->argument_count] =
                strdup(tokens[i].value);

            current->argument_count++;

            i++;
        }

        /* PIPE */
        else if (tokens[i].type == PIPE)
        {
            if (current->argument_count == 0)
            {
                printf("Syntax Error: Empty command before pipe.\n");
                free_commands(head);
                return NULL;
            }

            if (i + 1 >= count || tokens[i + 1].type == PIPE ||

```

```

        tokens[i + 1].type == END)
    {
        printf("Syntax Error: Empty command after pipe.\n");
        free_commands(head);
        return NULL;
    }

    current->next = create_command();
    current = current->next;

    i++;
}

/* INPUT REDIRECTION */
else if (tokens[i].type == INPUT)
{
    if (i + 1 >= count || tokens[i + 1].type != WORD)
    {
        printf("Syntax Error: Input file expected.\n");
        free_commands(head);
        return NULL;
    }

    current->input_file = strdup(tokens[i + 1].value);

    i += 2;
}

/* OUTPUT REDIRECTION */
else if (tokens[i].type == OUTPUT ||
        tokens[i].type == APPEND)
{
    if (i + 1 >= count || tokens[i + 1].type != WORD)
    {
        printf("Syntax Error: Output file expected.\n");
        free_commands(head);
        return NULL;
    }

    current->output_file = strdup(tokens[i + 1].value);

    if (tokens[i].type == APPEND)
    {
        current->append = 1;
    }

    i += 2;
}

/* BACKGROUND */
else if (tokens[i].type == BACKGROUND)
{
    current->background = 1;
    i++;
}

else
{
    printf("Syntax Error.\n");
    free_commands(head);
    return NULL;
}
}

return head;
}

/* Display execution structure */
void display_structure(Command *head)
{
    int command_number = 1;

    printf("\n=== Execution Structure ===\n");

    while (head != NULL)
    {
        printf("\nCommand %d:\n", command_number);

        printf("Arguments: ");

        for (int i = 0; i < head->argument_count; i++)

```

```

    {
        printf("[%s] ", head->arguments[i]);
    }

    printf("\n");

    if (head->input_file != NULL)
    {
        printf("Input: %s\n", head->input_file);
    }

    if (head->output_file != NULL)
    {
        if (head->append)
            printf("Output: %s (append)\n", head->output_file);
        else
            printf("Output: %s\n", head->output_file);
    }

    if (head->background)
    {
        printf("Background: YES\n");
    }

    if (head->next != NULL)
    {
        printf("Pipe: YES\n");
    }

    head = head->next;
    command_number++;
}
}

int main()
{
    char input[500];
    Token tokens[MAX_TOKENS];

    printf("=== Simple Shell Parser ===\n");
    printf("Enter a command:\n\n");

    printf("shell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Error: Empty command.\n");
        return 0;
    }

    int count = tokenize(input, tokens);

    printf("\n=== Token Stream ===\n");

    for (int i = 0; i < count; i++)
    {
        printf("%s : %s\n",
            token_name(tokens[i].type),
            tokens[i].value);

        if (tokens[i].type == END)
            break;
    }

    Command *commands = parse(tokens, count);

    if (commands != NULL)
    {
        display_structure(commands);
        free_commands(commands);
    }

    return 0;
}

```

```

adithya@LAPTOP-07A78KDI:~/OS$ nano parser.c
adithya@LAPTOP-07A78KDI:~/OS$ gcc -Wall -Wextra parser.c -o parser
adithya@LAPTOP-07A78KDI:~/OS$ ./parser
=== Simple Shell Parser ===
Enter a command:

shell> ls -l

=== Token Stream ===
WORD : ls
WORD : -l

=== Execution Structure ===

Command 1:
Arguments: [ls] [-l]
adithya@LAPTOP-07A78KDI:~/OS$ |

```

```

adithya@LAPTOP-07A78KDI:~/OS$ ./parser
=== Simple Shell Parser ===
Enter a command:

shell> cat < input.txt

=== Token Stream ===
WORD : cat
INPUT : <
WORD : input.txt

=== Execution Structure ===

Command 1:
Arguments: [cat]
Input: input.txt
adithya@LAPTOP-07A78KDI:~/OS$ |

```

```

adithya@LAPTOP-07A78KDI:~/OS$ ./parser
=== Simple Shell Parser ===
Enter a command:

shell> ls > output.txt

=== Token Stream ===
WORD : ls
OUTPUT : >
WORD : output.txt

=== Execution Structure ===

Command 1:
Arguments: [ls]
Output: output.txt
adithya@LAPTOP-07A78KDI:~/OS$ |

```

```

adithya@LAPTOP-07A78KDI:~/OS$ ./parser
=== Simple Shell Parser ===
Enter a command:

shell> echo hello >> output.txt

=== Token Stream ===
WORD : echo
WORD : hello
APPEND : >>
WORD : output.txt

=== Execution Structure ===

Command 1:
Arguments: [echo] [hello]
Output: output.txt (append)
adithya@LAPTOP-07A78KDI:~/OS$ |

```